Fundamentos de Big Data y Ciencia de Datos

Tema 3. Arquitecturas Big Data

Índice

Esquema

Ideas clave

3.1. Introducción y objetivos

3.2. Batch y streaming

3.3. Introducción a Hadoop

3.4. Ingesta y procesamiento de datos

3.5. Almacenamiento de datos

3.6. Análisis y visualización de datos

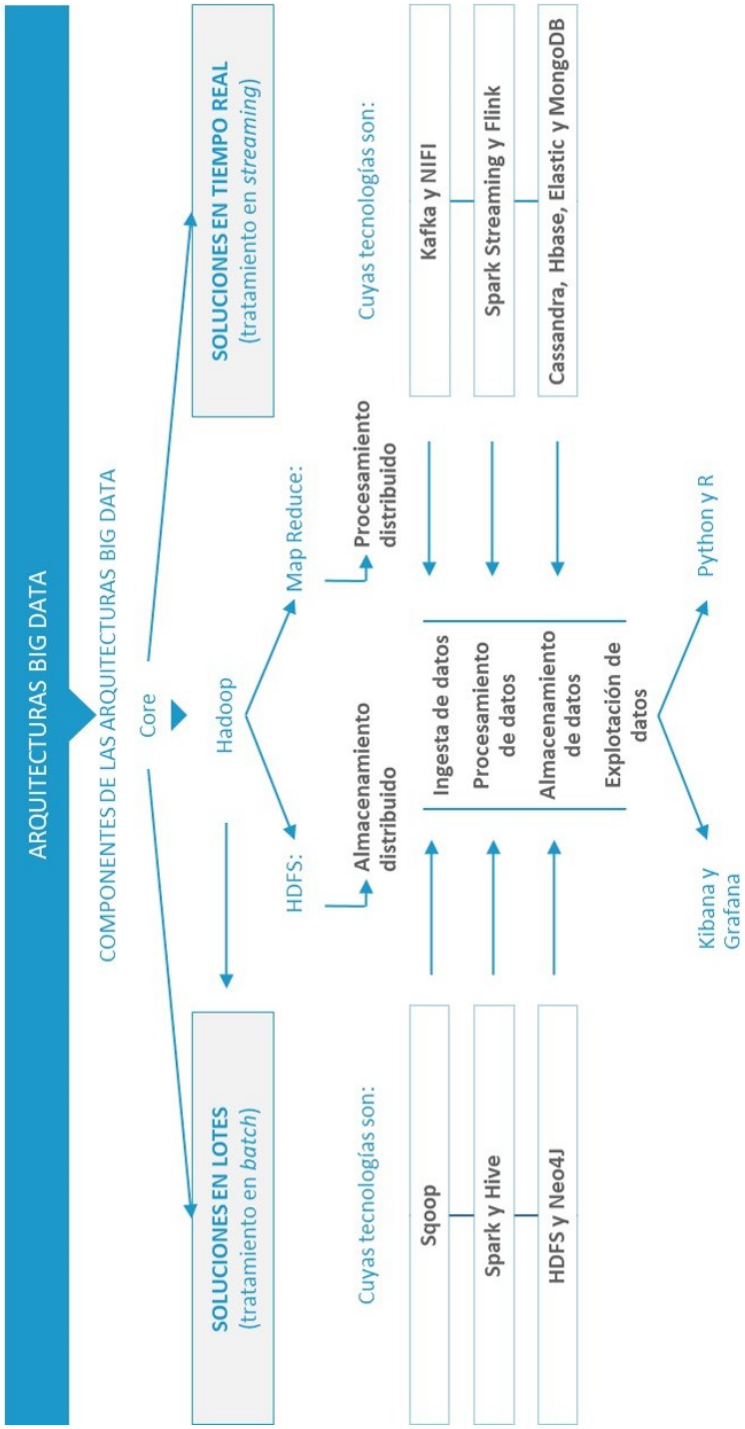
3.7. Referencias bibliográficas

A fondo

Big Data batch vs. streaming

Python vs. R

Esquema



© Universidad Internacional de La Rioja (UNIR)

3.1. Introducción y objetivos

En la actualidad, las organizaciones enfrentan el desafío de gestionar, procesar y analizar volúmenes masivos de datos generados a gran velocidad desde diversas fuentes. Estas arquitecturas de Big Data permiten procesar tanto grandes lotes de **datos almacenados** *(batch)* como **flujos continuos de datos en tiempo real** *(streaming),* lo que permite a las empresas tomar decisiones más rápidas e informadas.

El tema de **arquitecturas Big Data** aborda las principales soluciones, herramientas y estrategias necesarias para implementar, gestionar y optimizar infraestructuras capaces de soportar estos volúmenes de datos. Se proporcionará una visión integral de los elementos clave que componen una arquitectura de Big Data moderna, cubriendo desde la **ingesta y el procesamiento de datos** hasta su **almacenamiento, análisis y visualización.**

Los estudiantes aprenderán sobre las diferencias y casos de uso para soluciones *batch* y *streaming,* los mecanismos de ingesta de datos y cómo optimizar el procesamiento para maximizar la eficiencia. También explorarán las distintas **opciones de almacenamiento** y cómo integrar **tecnologías de análisis y visualización** que permitan extraer valor de los datos.

Los objetivos perseguidos en este tema son los siguientes:

- Comprender las diferencias entre los enfoques *batch* y streaming para el procesamiento de datos masivos.
- Conocer las principales estrategias y tecnologías para la **ingesta y procesamiento** de datos en arquitecturas distribuidas.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- Explorar las soluciones de **almacenamiento** que soportan grandes volúmenes de datos, tanto estructurados como no estructurados, y aprender a elegir la más adecuada según el caso de uso.
- Desarrollar competencias en herramientas de **análisis y visualización** de datos para extraer *insights* accionables que guíen la toma de decisiones en tiempo real.

© Universidad Internacional de La Rioja (UNIR)

3.2. Batch y streaming

En los entornos de Big Data es esencial tratar grandes volúmenes de información que se generan desde diversas fuentes. Dependiendo de la naturaleza y la velocidad de los datos, existen dos **enfoques** principales para el **procesamiento:** *batch* y *streaming.* Estos enfoques permiten a las organizaciones manejar de manera eficiente tanto datos históricos como aquellos que se generan en tiempo real (Kunigk *et al.,* 2018).

También es necesario diferenciar las distintas **capas** que componen las arquitecturas de datos:

- Ingesta, para la captura de los datos.
- Procesamiento, para el tratamiento.
- Almacenamiento, para su almacenamiento.
- Explotación, para el consumo, análisis o accionamiento.

Batch

El procesamiento *batch* se caracteriza por manejar **grandes bloques de datos** que se procesan en intervalos específicos. Este enfoque es ideal cuando los datos no necesitan ser analizados en tiempo real y pueden acumularse para ser procesados luego en lotes.

Características

- **Frecuencia:** los datos se procesan en intervalos programados (diarios, semanales, mensuales).
- **Ejemplos:** cargas diarias de *data warehouses,* generación de informes periódicos de negocio o procesamiento de grandes conjuntos históricos.

© Universidad Internacional de La Rioja (UNIR)

Ventajas de soluciones

- Capacidad para procesar grandes volúmenes de datos acumulados.
- Es eficiente para tareas que no requieren resultados inmediatos.
- Permite reprocesar datos históricos o corregir errores en lotes.

Streaming

El procesamiento *streaming,* por otro lado, se enfoca en manejar datos en tiempo real a medida que se generan. Este enfoque es adecuado para situaciones donde se necesita tomar decisiones rápidamente o reaccionar a eventos en el momento en que ocurren.

Características

- **Frecuencia:** los datos se procesan inmediatamente al ser generados.
- **Ejemplos:** monitoreo de sensores IoT, análisis de fraudes en transacciones, actualización en tiempo real de datos de marketing.

Ventajas de las soluciones

- Tratamiento en tiempo real, permitiendo respuestas inmediatas.
- Evita la acumulación de datos no procesados.
- Ideal para casos de uso críticos donde la latencia o tiempo de respuesta debe ser mínima.

En muchos casos, las organizaciones necesitan una combinación de ambos enfoques. **La arquitectura Lambda** combina los procesamientos *batch* y s*treaming* para ofrecer una visión completa de los datos. Mientras el *batch* se encarga del procesamiento de grandes volúmenes de datos históricos, el *streaming* permite la respuesta en tiempo real a eventos recientes.

© Universidad Internacional de La Rioja (UNIR)

En la siguiente figura podemos ver cómo se integran las distintas capas de una arquitectura Big Data:

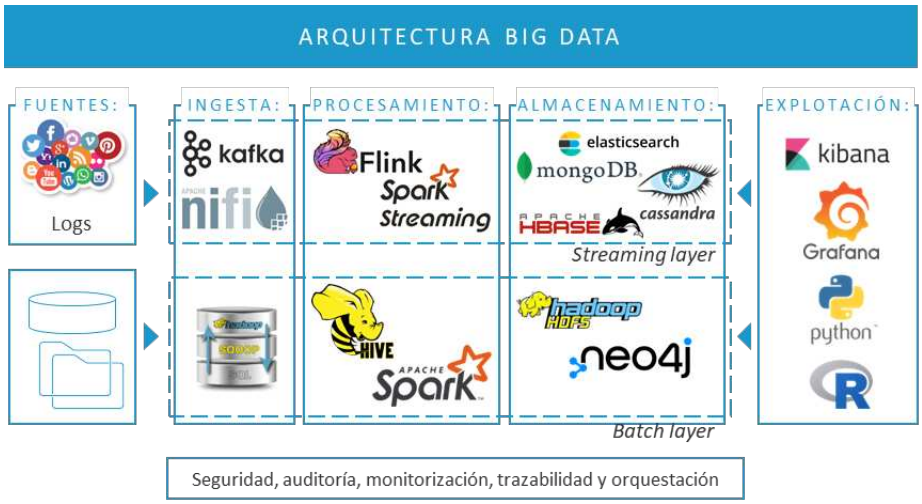
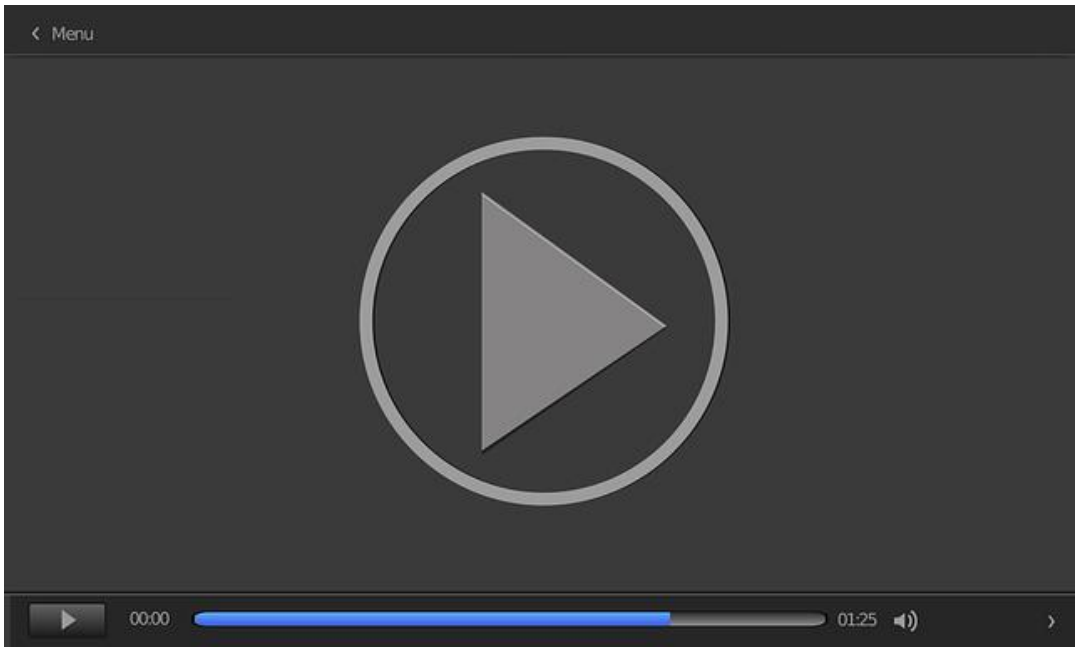


Figura 1. Arquitecturas Big Data. Fuente: elaboración propia

© Universidad Internacional de La Rioja (UNIR)

En el siguiente vídeo, *Introducción a las arquitecturas Big Data,* se realizará una breve introducción a las distintas fases de una arquitectura Big Data y ejemplos de algunas tecnologías.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=d0804992-d4c1-455e-9e9f-b23300f939b7

Antes de comenzar detallando cada una de estas tecnologías vamos a profundizar un poco más en **Hadoop.**

© Universidad Internacional de La Rioja (UNIR)

3.3. Introducción a Hadoop

Hadoop

Hadoop es un *framework* de *software* de código abierto diseñado para el almacenamiento y procesamiento distribuido de grandes volúmenes de datos en clusters de computadoras. Desarrollado inicialmente por Doug Cutting y Mike Cafarella en 2006 y gestionado por la Apache Software Foundation, Hadoop se ha convertido en una de las tecnologías más utilizadas en el ecosistema de Big Data (White, 2015).

Componentes principales de Hadoop

Hadoop *distributed file system* (HDFS)

HDFS es un sistema de archivos distribuido que permite almacenar grandes conjuntos de datos en bloques distribuidos en múltiples nodos de un clúster. Estos **bloques de datos** se replican en varios nodos para garantizar la tolerancia a fallos. HDFS está diseñado para manejar grandes cantidades de datos de manera eficiente y puede escalar fácilmente a cientos o miles de nodos.

MapReduce

MapReduce es un modelo de programación para el procesamiento de datos distribuidos que consiste en dos fases:

- *Map:* los datos de entrada se dividen en pequeños fragmentos (o bloques) y se procesan en paralelo por diferentes nodos del clúster.
- *Reduce:* los resultados de la fase de *Map* se combinan y se procesan para obtener el resultado final.

© Universidad Internacional de La Rioja (UNIR)

Este modelo es ideal para tareas que requieren el procesamiento de grandes cantidades de datos, como el análisis de *logs* o la minería de datos, por indicar algunos.

YARN *(yet another resource negotiator)*

YARN es el **sistema de gestión de recursos y planificación de tareas** de Hadoop. Permite a múltiples aplicaciones compartir los recursos del clúster, coordinando el uso de CPU, memoria y otras capacidades del sistema de forma eficiente.

Características clave de Hadoop

Almacenamiento distribuido

Hadoop divide los grandes conjuntos de datos en bloques y los almacena en múltiples nodos de un clúster, lo que permite un almacenamiento escalable y eficiente.

Procesamiento paralelo

Gracias al modelo MapReduce, Hadoop puede procesar datos en paralelo en diferentes nodos del clúster, lo que acelera el procesamiento de grandes volúmenes de datos.

Escalabilidad

Hadoop está diseñado para ser escalable horizontalmente, lo que significa que se pueden añadir fácilmente más nodos al clúster para manejar cantidades crecientes de datos.

© Universidad Internacional de La Rioja (UNIR)

Usos y aplicaciones de Hadoop

Análisis de datos masivos

Hadoop es utilizado para analizar grandes volúmenes de datos estructurados y no estructurados en empresas de todos los sectores.

Procesamiento de *logs* y análisis de eventos

Empresas como Facebook y X utilizan o utilizaban Hadoop para analizar grandes volúmenes de datos generados por sus sistemas en tiempo real.

Minería de datos

Es muy útil para descubrir patrones y relaciones ocultas en grandes conjuntos de datos.

© Universidad Internacional de La Rioja (UNIR)

3.4. Ingesta y procesamiento de datos

Comenzaremos entendiendo las diversas fuentes que se pueden cargar en las plataformas de datos y que las diferenciaremos según su naturaleza:

- **Estructuradas:** información principalmente tabular o relacional.
- **No estructuradas o semi estructuradas:** información que no sigue una estructura tabular o columnar.

Tipos de datos	Ejemplo
Estructurados	▸ Ficheros de texto con separadores o CSV ▸ Bases de datos relacionales: CRM y ERP ▸ Gestor de Campañas
No estructurados o semi estructurados	▸ Documentos de textos ▸ Facturas ▸ Texto libre de los call center ▸ Redes sociales ▸ Imágenes

Tabla 1. Ejemplos de tipos de datos. Fuente: elaboración propia.

Ingesta de Datos

La primera fase es la ingesta o captura de los datos que se realizará con diversas tecnologías según la naturaleza y tipología de los datos a cargar en nuestras plataformas de datos.

© Universidad Internacional de La Rioja (UNIR)

Kafka

Es una plataforma distribuida de transmisión de eventos (*streaming* de datos) que se utiliza para construir sistemas de mensajería altamente escalables, de alto rendimiento y tolerantes a fallos. Fue desarrollado originalmente por LinkedIn y luego se convirtió en un proyecto de **código abierto** mantenido por la Apache Software Foundation (Apache Kafka, s.f.).

Características:

- **Distribución:** Kafka está diseñado para funcionar en clústeres, lo que significa que los datos se replican y distribuyen entre varios nodos, lo que proporciona alta disponibilidad y tolerancia a fallos.
- **Alto rendimiento y escalabilidad:** Kafka puede manejar grandes volúmenes de datos y alta cantidad de eventos por segundo, lo que lo hace ideal para entornos con cargas de trabajo intensivas.
- **Durabilidad y persistencia:** los mensajes en Kafka se almacenan de manera duradera en el disco y pueden ser consumidos varias veces por diferentes consumidores.

Componentes:

- **Productores:** son los responsables de enviar los datos (mensajes) a Kafka.
- **Consumidores:** son las aplicaciones que leen y procesan los datos desde Kafka.
- **Tópicos:** los datos en Kafka se organizan en tópicos, que son los canales donde los productores envían mensajes y de donde los consumidores los leen.

© Universidad Internacional de La Rioja (UNIR)

NIFI

Es una herramienta de integración de datos de código abierto diseñada para automatizar y gestionar flujos de datos entre diferentes sistemas. Proporciona una interfaz gráfica de usuario que permite a los usuarios diseñar, supervisar y controlar el movimiento, transformación y manipulación de datos de manera eficiente y flexible. Fue desarrollado originalmente por la NSA (Agencia de Seguridad Nacional de EE. UU.) y luego donado a la Apache Software Foundation, donde se ha convertido en un proyecto popular dentro del ecosistema de Big Data (Apache Nifi, s.f.).

Características:

- **Flujos de datos en tiempo real y por lotes:** NIFI es capaz de manejar tanto flujos de datos en tiempo real *(streaming)* como datos en lotes *(batch),* lo que lo hace versátil para diferentes tipos de entornos.
- **Interfaz gráfica de usuario** (GUI): una de sus características más destacadas es su interfaz de diseño visual que permite a los usuarios arrastrar y soltar componentes para crear flujos de trabajo, sin necesidad de programación manual.
- **Soporte para múltiples fuentes y destinos:** NIFI se integra con una amplia variedad de sistemas y servicios, como bases de datos, API REST, sistemas de archivos, mensajería (Kafka, JMS), almacenamiento en la nube (Amazon S3, Google Cloud), entre otros.

© Universidad Internacional de La Rioja (UNIR)

Sqoop

Herramienta de código abierto utilizada para transferir grandes cantidades de datos entre sistemas de almacenamiento estructurados, como bases de datos relacionales (RDBMS) y plataformas de Big Data como Apache Hadoop y Apache Hive. Su nombre proviene de la combinación de **SQL to Hadoop** y está diseñada específicamente para facilitar la transferencia eficiente de datos entre sistemas tradicionales y entornos de Big Data (Apache Sqoop, 2021).

Características:

- **Transferencia de datos bidireccional:**
  - **Importación:** Sqoop permite importar datos desde bases de datos relacionales (como MySQL, Oracle, PostgreSQL, SQL Server, etc.) a sistemas de almacenamiento distribuidos como HDFS (Hadoop *distributed file system*), Hive o HBase.
  - **Exportación:** también permite exportar datos desde HDFS, Hive o HBase de vuelta a las bases de datos relacionales.
- **Compatibilidad con diversas bases de datos:** Sqoop es compatible con la mayoría de los sistemas de bases de datos relacionales que usan el protocolo JDBC. Los usuarios pueden conectarse fácilmente a estas bases de datos y mover los datos de manera segura.
- **Integración con Hive y HBase:** Sqoop facilita la importación de datos directamente a Hive (para consultas SQL sobre datos no estructurados) o a HBase (para almacenamiento distribuido basado en columnas).

© Universidad Internacional de La Rioja (UNIR)

Una vez capturados los datos serán susceptibles de procesar o almacenar según el tipo de dato y el tratamiento que se necesite realizar con este. Normalmente se procesa, tras su ingesta, en soluciones en *streaming* y se almacena cuando es necesario su procesamiento en *batch*.

Procesamiento de datos

Spark Streaming

Es un **componente** de Apache Spark diseñado para el procesamiento de flujos de datos en tiempo real. Permite la ingesta, el procesamiento y el análisis de datos que llegan de forma continua desde fuentes en tiempo real como sensores, *logs* de servidores, redes sociales o sistemas de mensajería (por ejemplo, Kafka o NIFI). Spark Streaming amplía la funcionalidad de Spark, que es una potente plataforma de procesamiento distribuido de datos en memoria, para manejar flujos de datos en tiempo real, de manera similar a como lo haría con datos estáticos o por lotes (Spark Streaming, s.f.).

Características:

- Micro-*batch*ing*:* a diferencia de otras soluciones de procesamiento en tiempo real, que procesan cada evento individualmente, Spark Streaming usa un enfoque llamado micro-*batch*ing*.* Los datos que llegan en un flujo continuo se dividen en pequeños lotes (por ejemplo, cada segundo) y luego se procesan como si fueran mini lotes de datos por lotes tradicionales.

- **Integración** con Apache Spark: Spark Streaming se integra perfectamente con el núcleo de Apache Spark, lo que permite aprovechar las mismas API y el modelo de procesamiento distribuido para ambos tipos de datos (en tiempo real y por lotes).

© Universidad Internacional de La Rioja (UNIR)

- **Compatibilidad con múltiples fuentes de datos:** Spark Streaming puede recibir datos de diversas fuentes, como Apache Kafka, Apache Flume, *sockets* TCP, sistemas de archivos como HDFS y S3 y, también, se integra con bases de datos y almacenes de datos, como se puede ver en la siguiente imagen.

Figura 2. Integración de Spark Streaming. Fuente: Spark Streaming, s. f.

Flink

Es una plataforma de procesamiento de flujos de datos distribuida y de código abierto, diseñada para manejar tanto **procesamiento de datos en tiempo real** *(streaming)* como **procesamiento por lotes** (*batch*). Es una de las principales herramientas utilizadas en el ecosistema de Big Data, especialmente para aplicaciones que requieren el procesamiento de grandes volúmenes de datos en tiempo real, como análisis de eventos, detección de fraudes, monitorización de sistemas y más (Flink, s.f.).

Características:

- **Escalabilidad y rendimiento:** Flink está diseñado para escalar horizontalmente y manejar grandes volúmenes de datos, distribuyendo el procesamiento en múltiples nodos en un clúster. Su motor de procesamiento optimiza automáticamente las tareas para maximizar el rendimiento.

© Universidad Internacional de La Rioja (UNIR)

- **Procesamiento de ventanas:** Flink proporciona una robusta API para procesamiento basado en ventanas de tiempo, lo que permite agrupar eventos en ventanas de tiempo específicas (por ejemplo, cada 5 minutos) y realizar cálculos sobre estos subconjuntos de datos. Esto es especialmente útil para aplicaciones en tiempo real que necesitan agrupar datos durante intervalos de tiempo.

- **Integración con otras tecnologíasde Big Data:** Flink se integra con una amplia gama de sistemas de almacenamiento y fuentes de datos, incluyendo Apache Kafka, Apache Hadoop, HDFS, Cassandra, y muchos otros. Esto permite que Flink se use en arquitecturas complejas y conectadas.

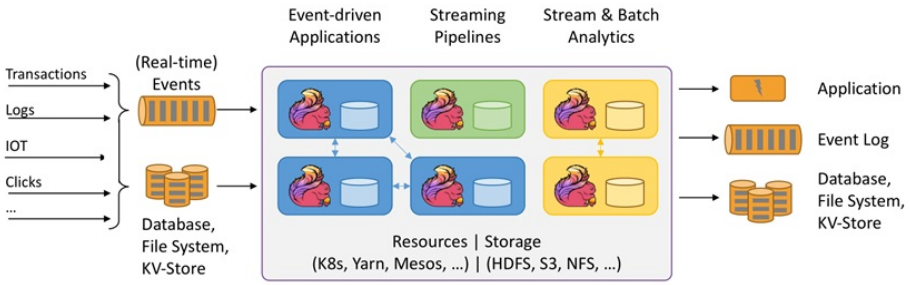


Figura 3. Arquitectura. Fuente: Fernández, 2024.

Continuaremos con las soluciones de procesamiento en Batch.

© Universidad Internacional de La Rioja (UNIR)

Spark

Plataforma de procesamiento de datos de código abierto diseñada para realizar análisis rápidos y distribuidos de grandes volúmenes de datos. Fue desarrollado inicialmente en el laboratorio AMPLab de la Universidad de California, Berkeley, y es ahora uno de los proyectos más importantes de la Fundación Apache (Apache Spark, s.f.).

Características:

- **Compatibilidad con múltiples lenguajes:** Spark admite varios lenguajes de programación como **Java, Scala, Python y R.** Esto permite a los desarrolladores utilizar el lenguaje que mejor se adapte a sus habilidades o al entorno en el que trabajan.

- **Capacidades avanzadas:** Spark incluye una variedad de capacidades para diferentes tipos de análisis:

  - Spark SQL: para consultas estructuradas utilizando SQL, DataFrames y Datasets.
  - MLlib: para algoritmos de *machine learning* (aprendizaje automático).
  - GraphX: para análisis de grafos.
  - Spark Streaming: para procesamiento de flujos de datos en tiempo real que hemos visto anteriormente.

- **Integración con Hadoop:** aunque Spark no depende de Hadoop, puede integrarse fácilmente con el ecosistema Hadoop, utilizando HDFS (Hadoop *distributed file system*) para almacenamiento de datos y Yarn como gestor de recursos.

© Universidad Internacional de La Rioja (UNIR)

Hive

Herramienta de *software* de código abierto diseñada para consultar y gestionar grandes conjuntos de datos almacenados en sistemas distribuidos, como el HDFS. Hive fue desarrollado inicialmente por Facebook y ahora es parte del ecosistema de Apache Hadoop. Su principal objetivo es proporcionar una interfaz similar a SQL para que los usuarios puedan realizar consultas sobre grandes volúmenes de datos almacenados en Hadoop, sin necesidad de aprender programación avanzada en MapReduce o utilizar API complejas (Apache Hive, s. f.).

Características:

- **Lenguaje SQL** (HiveQL): Hive utiliza un lenguaje de consulta similar a SQL llamado HiveQL (Hive *query language*), lo que facilita el uso de Hadoop a usuarios con experiencia en bases de datos relacionales. Esto permite ejecutar consultas tipo SQL para manipular y analizar datos almacenados en Hadoop, facilitando la interacción con grandes volúmenes de información.

- **Compatibilidad con HDFS:** Hive se integra con Hadoop y utiliza HDFS para almacenar los datos. Además, puede trabajar con otros sistemas de almacenamiento distribuidos como Apache HBase, Amazon S3 o Azure Blob Storage, lo que le otorga flexibilidad en entornos de Big Data.

- **Soporte para múltiples formatos de datos:** Hive puede trabajar con diversos formatos de datos, incluyendo texto plano, CSV, JSON, ORC *(optimized row columnar),* Parquet y Avro, lo que facilita su uso en diversos tipos de aplicaciones de análisis de datos.

© Universidad Internacional de La Rioja (UNIR)

3.5. Almacenamiento de datos

En cuanto al almacenamiento, existe una amplia variedad de soluciones que pueden utilizarse según las necesidades de los diversos casos de uso. Esto complica la elección y a menudo es necesario combinar varias opciones según aumenta la complejidad o el número de proyectos.

Las bases de datos NoSQL (*not only* SQL) representan un enfoque diferente a las bases de datos relacionales tradicionales, siendo especialmente adecuadas para entornos de Big Data por su **escalabilidad y rendimiento.** Sin embargo, requieren un enfoque distinto en la modelación de datos para maximizar su potencial y aprovechar al máximo estas tecnologías.

Antes de adentrarnos en este tipo de soluciones revisaremos el **teorema CAP** o **teorema Brewer** (IBM, s. f.), el cual dice que en sistemas distribuidos es **imposible garantizar a la vez consistencia, disponibilidad y tolerancia a particiones** *(consistency, availability, partition and tolerance).* Veamos qué son estas características:

- **Consistencia:** al realizar una consulta o inserción siempre se tiene que recibir la misma información, con independencia del nodo o servidor que procese la petición.
- **Disponibilidad:** que todos los clientes puedan leer y escribir, aunque se haya caído uno de los nodos.
- **Tolerancia a particiones:** a veces traducido como tolerancia a **fallos,** aunque ello presta a confusión con el punto anterior. Los sistemas distribuidos pueden estar divididos en particiones (generalmente de forma geográfica), así que esta condición implica que el sistema tiene que seguir funcionando, aunque existan fallos o caídas parciales que dividan el sistema.

© Universidad Internacional de La Rioja (UNIR)

En la Figura 4 podemos ver cómo se organizan algunas de las bases de datos según las condiciones que cumplen el teorema CAP:

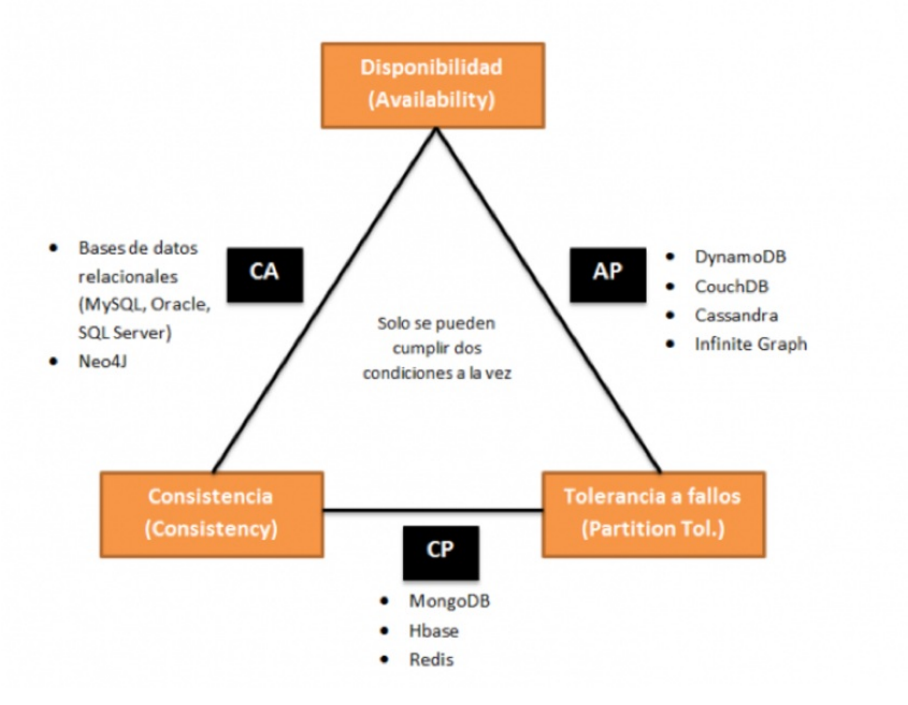


Figura 4. Clasificación de las bases de datos según el teorema CAP. Fuente: Fernández, 2017.

- **AP:** garantizan la **disponibilidad y tolerancia** a particiones, pero no la consistencia, al menos de forma total. Algunas de ellas consiguen una consistencia parcial a través de la replicación y la verificación.
- **CP:** garantizan **consistencia y tolerancia a particiones.** Para lograr la consistencia y replicar los datos a través de los nodos, sacrifican la disponibilidad.
- **CA:** garantizan **consistencia y disponibilidad,** pero tienen problemas con la tolerancia a particiones. Este problema lo suelen gestionar replicando los datos.

Hay que tener en cuenta que esta clasificación no es definitiva, ya que algunos de estos sistemas NoSQL pueden configurarse para cambiar su comportamiento.

© Universidad Internacional de La Rioja (UNIR)

Por tanto, además de pensar en el tipo de base de datos NoSQL que mejor se adapta a nuestro modelo de datos, también tendremos que pensar en su funcionamiento. Así, podremos conseguir que nuestra aplicación funcione de la mejor manera posible.

En la siguiente imagen podemos ver los **cuatro tipos de bases de datos NoSQL** que hay partiendo de las bases de datos tradicionales:

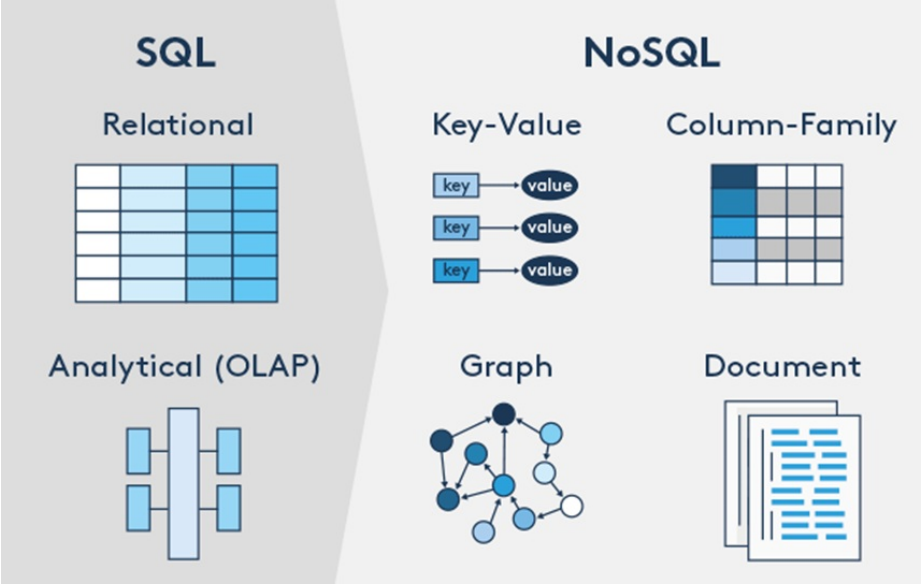


Figura 5. Tipos de bases de datos no relacionales. Fuente: Macarrón, 2021.

Vamos a ver algunas de las soluciones que más se están utilizando en las empresas hoy en día, dando cobertura a diversas aplicaciones y casos de uso.

© Universidad Internacional de La Rioja (UNIR)

Elasticsearch

Es un motor de búsqueda y análisis de código abierto, altamente escalable y distribuido, diseñado para almacenar, buscar y analizar grandes volúmenes de datos en tiempo real. Está basado en la tecnología de Apache Lucene y permite realizar **búsquedas rápidas,** además de ofrecer **análisis complejos** a través de su capacidad de indexación avanzada.

Características

- **Búsqueda en tiempo real:** Elasticsearch permite realizar búsquedas casi instantáneas en grandes conjuntos de datos, lo que lo hace ideal para aplicaciones que requieren resultados rápidos, como sistemas de monitoreo y análisis en tiempo real.
- **Soporte para datos no estructurados:** Elasticsearch puede manejar diversos tipos de datos, incluidos texto, números, fechas y datos no estructurados como documentos y archivos multimedia. Esto le permite realizar análisis avanzados de múltiples tipos de datos.
- **Indexación avanzada:** a través de su potente motor de indexación, Elasticsearch permite organizar los datos de manera eficiente para optimizar las consultas y búsquedas. Esto es especialmente útil para búsquedas de texto completo, filtrado y agregaciones complejas.
- **Soporte de búsquedas geoespaciales:** Elasticsearch tiene capacidades integradas para búsquedas y análisis geoespaciales, permitiendo trabajar con datos de ubicación y mapas.

© Universidad Internacional de La Rioja (UNIR)

Casos de uso

- **Análisis** de *logs* y **monitoreo:** Elasticsearch es utilizado junto con Logstash y Kibana (ELK Stack) para recopilar, analizar y visualizar *logs* de servidores y aplicaciones.
- **Comercio electrónico** y recomendaciones: las tiendas en línea utilizan Elasticsearch para búsquedas de productos y recomendaciones personalizadas.
- **Visualización de datos geoespaciales:** Elasticsearch permite indexar y consultar datos geoespaciales, lo que lo hace ideal para aplicaciones como mapas interactivos o seguimiento en tiempo real de vehículos.

Cassandra

Es una **base de datos orientada a columnas,** diseñada para manejar grandes cantidades de datos a través de múltiples servidores, proporcionando alta disponibilidad, escalabilidad y tolerancia a fallos sin un único punto de fallo. Fue desarrollada inicialmente por Facebook y luego se convirtió en un proyecto de código abierto gestionado por la Apache Software Foundation (Apache Cassandra, s.f.).

Características

- **Modelo de datos orientado a columnas:** a diferencia de las bases de datos relacionales, Cassandra organiza los datos en columnas, lo que permite acceder rápidamente a grandes volúmenes de datos y realizar operaciones eficientes de lectura y escritura.
- **Lecturas y escrituras rápidas:** Cassandra está optimizada para operaciones rápidas de lectura y escritura, lo que la hace adecuada para aplicaciones que requieren un alto rendimiento, como análisis en tiempo real, gestión de grandes volúmenes de datos y procesamiento de eventos en línea.

© Universidad Internacional de La Rioja (UNIR)

- **Lenguaje de consulta CQL** (Cassandra *query language*): Cassandra usa su **propio lenguaje de consulta,** CQL, que es similar a SQL, pero adaptado a su modelo de datos distribuido y orientado a columnas.

Casos de uso

- **Redes sociales:** manejo de grandes volúmenes de interacciones de usuarios.
- **Internet de las cosas** (IoT): almacenamiento de datos generados por sensores en tiempo real.
- **Comercio electrónico:** gestión de catálogos de productos y transacciones distribuidas.

Hbase

Base de datos de código abierto diseñada para manejar grandes volúmenes de datos estructurados distribuidos en clústeres de servidores. Se ejecuta sobre el HDFS y está diseñada para proporcionar **acceso aleatorio y en tiempo real a datos masivos,** lo cual es ideal para aplicaciones que requieren una gran cantidad de lectura y escritura (Apache HBase, s.f.).

Características

- **Modelo de datos orientado a columnas:** HBase almacena datos en formato de tabla con filas y columnas, pero a diferencia de las bases de datos relacionales tradicionales, está orientada a columnas, lo que permite la recuperación rápida de datos específicos.
- **Acceso aleatorio en tiempo real:** HBase está optimizado para proporcionar acceso en tiempo real a grandes volúmenes de datos, lo que lo hace adecuado para aplicaciones que necesitan **respuestas rápidas** y que no pueden permitirse latencias altas.

© Universidad Internacional de La Rioja (UNIR)

- **Compatibilidad con Hadoop:** HBase se integra estrechamente con Hadoop y el ecosistema de herramientas relacionadas, como MapReduce, Hive y Pig, lo que lo hace adecuado para entornos de **análisis de datos masivos,** donde se requiere procesamiento por lotes y en tiempo real.

Casos de uso

- **Aplicaciones de redes sociales:** maneja datos no estructurados generados por usuarios, como publicaciones, comentarios, y mensajes.
- **Sistemas de recomendación:** almacena y procesa grandes cantidades de datos de usuarios para generar recomendaciones personalizadas.
- **Almacenamiento de datos históricos:** utilizado en entornos que requieren almacenar grandes volúmenes de datos históricos, como registros financieros o de telecomunicaciones.

MongoDB

Es una base de datos **orientada a documentos,** diseñada para almacenar y gestionar grandes volúmenes de datos no estructurados o semiestructurados. A diferencia de las bases de datos relacionales tradicionales, MongoDB utiliza un modelo de datos basado en **documentos JSON** (BSON en su implementación) que permite una mayor flexibilidad y escalabilidad en el manejo de datos.

Características

- **Modelo orientado a documentos:** MongoDB almacena los datos en documentos en formato BSON (una forma binaria de JSON), que permite una estructura de datos más flexible. A diferencia de las tablas de las bases de datos relacionales, los documentos pueden tener diferentes campos y estructuras, lo que lo hace ideal para datos no estructurados o semiestructurados.

© Universidad Internacional de La Rioja (UNIR)

- **Esquema flexible:** no es necesario definir un esquema fijo antes de almacenar los datos, lo que permite una rápida iteración y modificación de la estructura de los datos según sea necesario.

- **Compatibilidad con diversas plataformas y lenguajes:** MongoDB es compatible con una amplia gama de plataformas y lenguajes de programación, lo que facilita su integración en diversos entornos tecnológicos.

Casos de uso

- **Aplicaciones web y móviles:** MongoDB es ampliamente utilizado en aplicaciones web y móviles que requieren almacenamiento flexible y escalable, como redes sociales, comercio electrónico y plataformas de *streaming.*

- **Internet de las cosas** (IoT): MongoDB es ideal para aplicaciones IoT que necesitan almacenar y gestionar datos generados por millones de dispositivos conectados.

Neo4J

Base de datos **orientada a grafos,** diseñada para modelar y gestionar relaciones complejas entre los datos. A diferencia de las bases de datos relacionales tradicionales, que almacenan datos en tablas y requieren uniones costosas para relacionar los datos, Neo4j utiliza **nodos y relaciones** (aristas) **para representar entidades y sus conexiones** de manera más directa y eficiente.

Características

- **Modelo de datos basado en grafos:** en lugar de tablas, Neo4j utiliza nodos para representar entidades (personas, productos, etc.) y relaciones (enlaces entre esas entidades) para modelar cómo están conectadas entre sí. Cada nodo y relación puede tener propiedades que almacenan información adicional.

© Universidad Internacional de La Rioja (UNIR)

- **Lenguaje de consulta Cypher:** Neo4j utiliza Cypher, un lenguaje de consulta diseñado específicamente para grafos, que permite realizar búsquedas y manipulaciones complejas de nodos y relaciones de manera fácil e intuitiva.

- **Alto rendimiento en consultas sobre relaciones:** Neo4j está optimizado para realizar consultas profundas sobre relaciones de datos, como encontrar todos los amigos de amigos de un usuario en una red social, identificar patrones en una red o descubrir conexiones ocultas entre entidades.

Casos de uso

- **Redes sociales:** Neo4j es ideal para modelar y consultar redes sociales, donde las conexiones entre personas, grupos, eventos y publicaciones pueden ser muy complejas.

- **Sistemas de recomendación:** muchas empresas usan Neo4j para analizar las relaciones entre usuarios, productos y preferencias, lo que permite crear recomendaciones personalizadas basadas en patrones de interacción.

- **Detección de fraudes:** Neo4j se utiliza para identificar patrones sospechosos en redes de transacciones financieras o logísticas, lo que facilita la detección de fraudes.

© Universidad Internacional de La Rioja (UNIR)

La Figura 5 muestra cómo se representa esta base de datos:

Figura 5. Análisis gráfico de Neo4j. Fuente: Sharma, 2019.

© Universidad Internacional de La Rioja (UNIR)

3.6. Análisis y visualización de datos

Para concluir, exploramos algunas de las **soluciones** disponibles para la explotación y el análisis de datos dentro de las plataformas de datos teniendo en cuenta que existen diversas herramientas en el mercado y solo se han seleccionado algunas soluciones *open source.*

Es esencial recordar que esta capa final de la arquitectura de datos da valor a todas las anteriores. A través de estas herramientas, los distintos usuarios podrán explotar o visualizar los datos, y en el área analítica, obtener resultados que respondan a las necesidades del negocio.

Visualización

Kibana

Herramienta de visualización y análisis de datos de código abierto diseñada para trabajar con Elasticsearch, que es un motor de búsqueda y análisis. Kibana permite a los usuarios explorar, visualizar y analizar grandes volúmenes de datos en tiempo real de manera intuitiva y accesible. A través de Kibana, se pueden crear *dashboards* (paneles de control) **dinámicos y gráficos interactivos** que ayudan a monitorear y entender los datos de una forma clara y rápida.

© Universidad Internacional de La Rioja (UNIR)

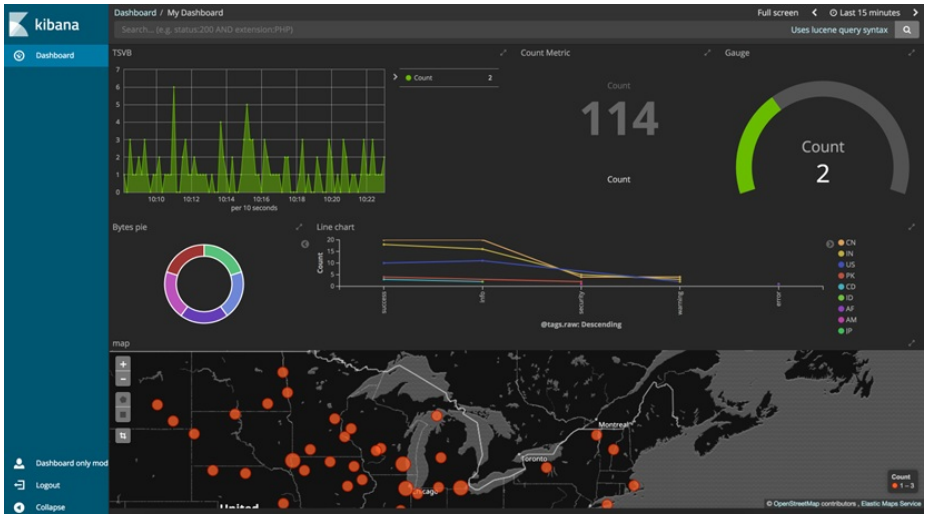


Figura 6. Ejemplos visuales de Elastic. Fuente: Gammon, 2017.

Grafana

Plataforma de código abierto para la visualización, monitoreo y análisis de datos en tiempo real, que permite a los usuarios crear paneles de control (dashboards) interactivos y altamente personalizables. Se utiliza principalmente para monitorear sistemas y aplicaciones, permitiendo a los equipos de operaciones e ingeniería rastrear métricas clave, detectar anomalías y obtener información sobre el rendimiento del sistema o infraestructura.

© Universidad Internacional de La Rioja (UNIR)

Figura 7. Ejemplo de la fuente e información adicional. Fuente: Grafana, s. f.

© Universidad Internacional de La Rioja (UNIR)

Análisis

Python

Lenguaje de programación de alto nivel, interpretado y de propósito general, conocido por su **simplicidad y legibilidad.** Fue creado por Guido van Rossum y lanzado por primera vez en 1991. Python permite a los programadores escribir código de manera eficiente, gracias a su sintaxis clara y concisa, lo que facilita tanto el desarrollo rápido de aplicaciones como la comprensión del código, incluso para principiantes.

Python es ampliamente utilizado en diversos **campos** como el desarrollo web, la automatización de tareas, el análisis de datos, la inteligencia artificial, el *machine learning,* y la ciencia de datos. Su extensa biblioteca estándar y la gran cantidad de bibliotecas de terceros lo hacen muy versátil, permitiendo a los desarrolladores trabajar con diferentes tipos de proyectos, desde análisis numérico hasta procesamiento de imágenes.

En el ámbito de la ciencia de datos y el Big Data, Python es uno de los lenguajes más utilizados, gracias a sus **bibliotecas especializadas** como NumPy, Pandas, Matplotlib, TensorFlow, y Scikit-learn, que facilitan tareas de análisis, visualización y modelado de datos.

© Universidad Internacional de La Rioja (UNIR)

R

R es un **entorno** de *software* especializado en el análisis estadístico y la visualización de datos. Fue desarrollado en los años 90 por Ross Ihaka y Robert Gentleman en la Universidad de Auckland, Nueva Zelanda. R es ampliamente utilizado en los campos de la estadística, la ciencia de datos, el análisis predictivo y la investigación científica debido a su capacidad para **manejar cálculos matemáticos complejos**.

La elección entre Python y R dependerá de diversos factores, puesto que, *a priori,* ambos se pueden utilizar para analítica avanzada de datos. Profundizando a fondo hay más diferencias que nos pueden ayudar en esta elección (Arsis, s.f.)

© Universidad Internacional de La Rioja (UNIR)

3.7. Referencias bibliográficas

Fernández, O. (2024, julio 14). *Apache Flink: Introducción.* Aprender Big Data. https://aprenderbigdata.com/introduccion-apache-flink/

Fernández, R. (2017, mayo 31). *NoSQL: clasificación de las bases de datos según el teorema CAP.* Genbeta. https://www.genbeta.com/desarrollo/nosql-clasificacion-de-las-bases-de-datos-segun-el-teorema-cap

Gammon, S. (2017, octubre 10). *Kibana Dashboard Only Mode.* Elastic. https://www.elastic.co/es/blog/kibana-dashboard-only-mode

Grafana OSS. (s. f.). En *Grafana.* https://grafana.com/oss/grafana/

Kunigk, J., Buss, I., Wilkinson, P. y George, L. (2018). *Architecting Modern Data Platforms: A Guide to Enterprise Hadoop at Scale.* O'Reilly.

IBM. (s. f.). *¿Qué es el teorema CAP?* https://www.ibm.com/es-es/topics/cap-theorem

Macarrón, P. (2021, marzo 8). *Tipos de bases de datos no relacionales.* Certia. https://www.certia.net/tipos-de-bases-de-datos-no-relacionales/

Spark Streaming. (s. f.). *Spark Streaming Programming Guide.* https://spark.apache.org/docs/latest/streaming-programming-guide.html

Sharma, A. (2019, abril 30). *Challenges of neo4j at the heart of software.* Dev. https://dev.to/l04db4l4nc3r/challenges-of-neo4j-at-the-heart-of-software-2dbi

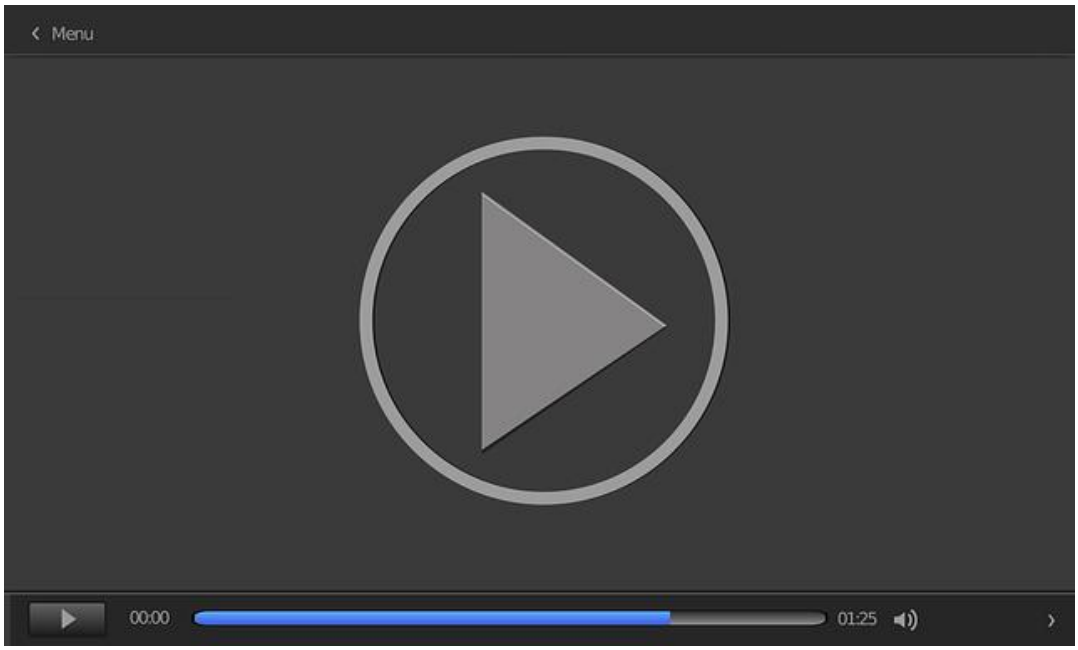
White, T. (2015). *Hadoop: The Definitive Guide* (4.ª ed.). O'Reilly. https://piazza-resources.s3.amazonaws.com/ist3pwd6k8p5t/iu5gqbsh8re6mj/OReilly.Hadoop.The.Definitive.Guide.4th.Edition.2015.pdf

© Universidad Internacional de La Rioja (UNIR)

Big Data batch vs. streaming

Cloudvala. (2019, marzo 2). *Big-data Batch vs streaming* [Vídeo]. YouTube. https://www.youtube.com/watch?v=9OUD2CVcoOg

El vídeo muestra la diferencia del procesamiento *batch* o por lotes y el procesamiento en tiempo real.



Accede al vídeo:

https://www.youtube.com/embed/9OUD2CVcoOg

© Universidad Internacional de La Rioja (UNIR)

Python vs. R

Fuentes, F. (2024, octubre 16). *Python vs. R: comparativa para los proyectos de ciencia de datos.* Arsys. https://www.arsys.es/blog/python-vs-r-comparativa-en-la-ciencia-de-datos

El artículo compara Python con R, dos lenguajes clave en la ciencia de datos. Python es un lenguaje multipropósito con una curva de aprendizaje sencilla, ideal para principiantes, y es ampliamente usado en diversas aplicaciones. Su flexibilidad y escalabilidad lo hacen adecuado para proyectos grandes. R, por otro lado, está diseñado específicamente para el análisis estadístico y la visualización de datos. Es más complejo de aprender, pero excelente para tareas específicas como el análisis estadístico y el aprendizaje automático. La elección entre Python o R depende del tipo de proyecto y las necesidades específicas de análisis.

© Universidad Internacional de La Rioja (UNIR)